Entrega 3 — Divide y Vencerás, Programación Dinámica y Comparativa Final

Caso de uso: Solver de Sudoku.

Juan Ocampo – Cristopher Sánchez

1. Solución con Divide y Vencerás y Programación Dinámica

a. Solución con Divide y Vencerás (src/divide_y_venceras.py)

Identificación de subproblemas y descomposición

El paradigma de Divide y Vencerás consiste en descomponer un problema en subproblemas más pequeños del mismo tipo, resolverlos de manera recursiva y combinar sus resultados para obtener la solución global. En un Sudoku de 9x9, aunque las celdas están acopladas por filas, columnas y cajas de 3x3, el algoritmo implementado en src/divide_y_venceras.py descompone geométricamente el tablero por rangos de filas y bandas horizontales (intervalos [inicio, fin) sobre las 9 filas):

- El tablero de 9 filas se divide en 3 bandas horizontales de 3 filas cada una (filas 0..2, 3..5 y 6..8), aprovechando que las cajas de 3x3 no cruzan los límites entre bandas.
- A su vez, cada banda de 3 filas se divide recursivamente hasta llegar a filas individuales (fin - inicio == 1), que constituyen el caso base.

Explicación de las fases: Dividir, Vencer y Combinar

La implementación en src/divide_y_venceras.py se estructura en cuatro componentes claros:

- **Caso Base (completar_fila(tablero, fila)):** Cuando el rango contiene una única fila (fin - inicio == 1), se identifican las columnas vacías de esa fila y los dígitos del 1 al 9 que faltan en ella. Mediante itertools.permutations(faltantes), se generan todas las formas de rellenar la fila que sean válidas respecto a las pistas iniciales del tablero (usando es_valido).
- **Fase DIVIDIR (resolver_filas):** Dado un intervalo de filas [inicio, fin) con más de una fila, el algoritmo calcula el punto de corte: medio = inicio + 3 if fin - inicio > 3 else inicio + 1. Si el bloque abarca más de una banda (por ejemplo

0..9 o 3..9), corta exactamente en la frontera de una banda de 3 filas; si está dentro de una misma banda (3 o 2 filas), separa la primera fila del resto.

- **Fase VENCER:** Se resuelven recursivamente y por separado ambos subproblemas: izquierda = resolver_filas(tablero, inicio, medio, ...) y derecha = resolver_filas(tablero, medio, fin, ...). Cada llamada devuelve la lista de todas las configuraciones parcialmente válidas para su respectivo subconjunto de filas.
- **Fase COMBINAR (hay_repetidos(filas, inicio)):** Se recorren los pares (filas_izq, filas_der) provenientes de ambas mitades, se concatenan (filas = filas_izq + filas_der) y se filtran mediante hay_repetidos(filas, inicio), conservando únicamente las uniones que no repiten dígitos en ninguna de las 9 columnas ni en las cajas de 3x3 compartidas.

Análisis de complejidad y relación con el Teorema Maestro

- **Complejidad temporal y Teorema Maestro:** En términos del número de filas F = 9 (dividiendo el tablero en subproblemas de filas), la recurrencia tiene la forma estructural $T(F) = 2 \cdot T(F/2) + O(|L| \cdot |R|)$, donde |L| y |R| son las cantidades de configuraciones parciales válidas del subproblema izquierdo y derecho que deben cruzarse en la fase de combinación. En el Teorema Maestro clásico ($T(m) = a \cdot T(m/b) + f(m)$), si cada fila tiene una única configuración válida (|L| = |R| = 1, como ocurre en nuestro caso_pequeno donde parciales_por_banda = [1, 1, 1]), el costo de combinación es constante O(1) y por el Teorema Maestro (a = 2, b = 2, f(F) = O(1)) el costo respecto a las filas es lineal O(F). Sin embargo, respecto al número de celdas vacías n, en el peor caso una fila con k celdas vacías genera hasta k! permutaciones válidas, y cruzar las soluciones parciales entre filas y bandas multiplica las combinaciones válidas. Por ello, en el peor caso el costo está dominado por la generación y combinación de permutaciones por fila, resultando en una complejidad exponencial/factorial O((k!)^9) (acotada en nuestra comparativa por max_combinaciones = 100,000).
  Complejidad espacial: La complejidad espacial es O(P), donde P es el número de configuraciones parciales válidas almacenadas simultáneamente.

b. Solución con Programación Dinámica (src/programacion_dinamica.py)

Definición del estado, subestructura óptima y solapamiento de subproblemas

En Divide y Vencerás se generan todas las combinaciones de un bloque sin recordar qué configuraciones de columnas ya fueron evaluadas desde arriba. Para

optimizar esto, en src/programacion_dinamica.py se implementó Programación Dinámica Top-Down con Memoización avanzando fila por fila (fila = 0, 1, ..., 8):

- **Preprocesamiento por fila:** Se precalculan las permutaciones válidas de cada fila f usando completar_fila(tablero, f).
- **Definición del Estado (estado = (fila, columnas, cajas)):** Para resolver el tablero desde la fila actual hasta la fila 8, no importa en qué orden exacto se colocaron los números en las filas anteriores (0..fila-1); el pasado queda completamente resumido por:
    a. fila: el índice de la fila actual (0 a 9).
    b. columnas: una tupla de 9 conjuntos inmutables (frozenset) con los dígitos ya usados en cada una de las 9 columnas.
    c. cajas: una tupla de 3 conjuntos (frozenset) con los dígitos usados en las 3 cajas 3x3 de la banda actual.
    Además, cada vez que empieza una banda nueva (fila % 3 == 0, en las filas 0, 3 y 6), las cajas se reinician a (frozenset(), frozenset(), frozenset()) porque las cajas 3x3 de la banda anterior ya quedaron cerradas.
- **Solapamiento de subproblemas y Memoización (memoria = {}):** Distintas elecciones de filas en la parte superior del tablero pueden utilizar exactamente el mismo conjunto acumulado de números por columna y caja. Cuando resolver_desde(fila, columnas, cajas) detecta que estado está en memoria, retorna el resultado almacenado en O(1) e incrementa el contador visitas["reutilizados"] sin reexplorar las filas restantes. En nuestras pruebas sobre caso_grande (n = 55), el algoritmo visitó 1,498 estados únicos y reutilizó 295 estados repetidos desde la memoria.

Análisis de complejidad de Programación Dinámica

- **Complejidad temporal:** Sea E el número de estados únicos alcanzables (fila, columnas, cajas) y P_f el número de permutaciones válidas para la fila f (con P_f ≤ k!). Gracias a la memoización, cada estado único se resuelve una sola vez probando las opciones de la fila actual y verificando su compatibilidad mediante cabe(opcion, columnas, cajas) en tiempo constante O(1). Por lo tanto, la complejidad temporal es O(E · P_f) (acotada en la comparativa por max_estados = 10,000).
- **Complejidad espacial:** El diccionario memoria almacena a lo sumo E estados únicos y la pila de recursión tiene profundidad fija de 9 niveles (una llamada por fila), por lo que la complejidad espacial es O(E).

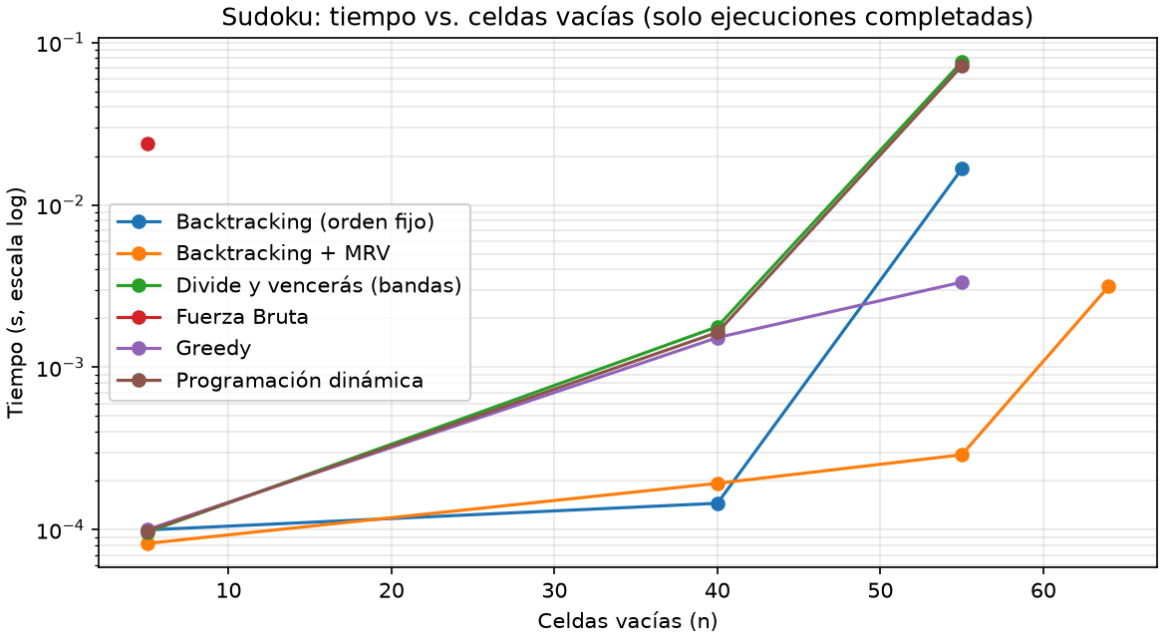
2. Comparativa Final

a. Tabla comparativa de las 5 técnicas aplicadas al problema

Nota sobre el tamaño de entrada (n): Dado que un Sudoku estándar 9x9 tiene 81 celdas fijas, el tamaño de entrada n corresponde al número de celdas vacías, evaluado en los 4 casos del proyecto: pequeño (n=5), mediano (n=40), grande (n=55) y extremo (n=64, 17 pistas).

Técnica	Complejidad Temporal	Complejidad Espacial	¿Solución óptima / completa?	Caso Pequeño (n=5)	Caso Mediano (n=40)	Caso Grande (n=55)	Caso Extremo (n=64)
Fuerza Bruta	O(9^n)	O(n)	Sí (teóricamente; inviable para n > 7)	0.02286 s (OK, 44,595 comb.)	0.09973 s (Excedió límite 100k)	0.12275 s (Excedió límite 100k)	0.14059 s (Excedió límite 100k)
Greedy	O(n²)	O(1)	No (incompleto, sin retroceso)	0.00009 s (OK)	0.00140 s (OK)	0.00308 s (OK)	0.00174 s (Sin solución)
Backtracking (orden fijo)	O(9^n) peor caso (con poda)	O(n)	Sí (sujeto a límite de nodos)	0.00008 s (OK, 6 nodos)	0.00014 s (OK, 83 nodos)	0.01539 s (OK, 21,622 nodos)	0.08425 s (Excedió límite 100k)
Backtracking + MRV	O(9^n) peor caso (poda + heurística)	O(n)	Sí (resolvió 4/4 casos)	0.00008 s (OK, 6 nodos)	0.00019 s (OK, 41 nodos)	0.00029 s (OK, 56 nodos)	0.00302 s (OK, 678 nodos)
Divide y Vencerás (bandas)	O((k!)^9) (combinación de bandas)	O(P) (parciales por banda)	Sí (en casos con n <= 55)	0.00009 s (OK, parciales [1,1,1])	0.00164 s (OK, parciales [1,9,4])	0.07056 s (OK, [405,6,52])	Omitido (n > 55)
Programación Dinámica	O(E · P_f) (estados memoizados)	O(E) (tabla memoria)	Sí (en casos con n <= 55)	0.00009 s (OK, 9 estados)	0.00150 s (OK, 13 estados)	0.06630 s (OK, 1498 est., 295 reut.)	Omitido (n > 55)

b. Gráfica de tiempos de ejecución de las soluciones



c. Análisis: ¿Qué técnica es la más adecuada para este problema y por qué?

La técnica más adecuada para resolver el problema del Sudoku es **Backtracking con la heurística MRV (Minimum Remaining Values)** y representación por máscaras de bits, por tres razones demostradas tanto en el análisis teórico como en los resultados experimentales:

1. **Propagación inmediata de restricciones globales:** El Sudoku es un Problema de Satisfacción de Restricciones (CSP) fuertemente acoplado donde cada celda afecta simultáneamente a su fila, su columna y su caja 3x3. Mientras que Divide y Vencerás y Programación Dinámica operan generando primero todas las permutaciones válidas de cada fila (lo cual se vuelve costoso cuando una fila tiene muchas celdas vacías), Backtracking + MRV restringe el dominio celda por celda en tiempo constante O(1) mediante operaciones bit a bit.
2. **Completitud frente a Greedy:** A diferencia del enfoque Greedy, que toma decisiones irrevocables y falla en el caso extremo (n = 64, 17 pistas) al quedarse sin candidatos en una bifurcación, Backtracking garantiza encontrar la solución única siempre que exista.
3. **Eficiencia superior en todos los tamaños de entrada:** Al elegir siempre la celda con menos candidatos disponibles (principio Fail-First), el factor de ramificación efectivo se reduce casi a 1. Esto permitió resolver los 4 casos de prueba en tiempos de entre 0.00008 s y 0.00302 s, superando por órdenes

de magnitud a Fuerza Bruta, Backtracking de orden fijo, Divide y Vencerás y Programación Dinámica. Asimismo, al comparar Divide y Vencerás contra Programación Dinámica en el caso grande (n = 55), Programación Dinámica resultó más rápida (0.06630 s frente a 0.07056 s) gracias a los 295 estados reutilizados por memoización.

3. Conclusiones

a. Reflexión sobre lo aprendido al aplicar cada técnica al mismo problema

Resolver el mismo problema de Sudoku a través de cinco paradigmas algorítmicos distintos demostró cómo el diseño del algoritmo y el aprovechamiento de las restricciones cambian radicalmente la viabilidad práctica de una solución:

- Pasamos de una solución de Fuerza Bruta que a partir de n = 8 celdas vacías tarda decenas de segundos (y que para n = 55 requeriría 9^55 operaciones, algo físicamente imposible), a una solución de Backtracking + MRV que resuelve tableros de 55 y 64 celdas vacías en menos de 4 milisegundos.
- Comprobamos que dos algoritmos con la misma cota superior teórica de peor caso O(9^n) (Fuerza Bruta, Backtracking de orden fijo y Backtracking + MRV) tienen comportamientos empíricos abismalmente distintos debido a la poda temprana y al ordenamiento heurístico de variables.
- Aprendimos a adaptar paradigmas como Divide y Vencerás (dividiendo el tablero en bandas horizontales) y Programación Dinámica (definiendo un estado resumen de columnas y cajas ocupadas por fila) a un problema de satisfacción de restricciones, verificando experimentalmente el solapamiento de subproblemas con 295 estados reutilizados en el caso grande.

b. ¿En qué escenarios reales usaríamos cada paradigma?

- **Fuerza Bruta:** Validación de casos de prueba pequeños, generación de oráculos de referencia para testing o problemas donde el espacio de búsqueda total es muy reducido (n ≤ 7).
- **Greedy:** Sistemas de respuesta en tiempo real con recursos muy limitados, heurísticas constructivas iniciales o preprocesamiento rápido para rellenar decisiones triviales antes de ejecutar un algoritmo exacto.
- **Backtracking (+ Heurísticas como MRV):** Planificación de horarios universitarios (timetabling), asignación de recursos y turnos de personal, configuración de sistemas, ruteo con restricciones y resolución exacta de puzzles y problemas NP-completos.

- **Divide y Vencerás:** Procesamiento distribuido o paralelizable donde las partes del problema pueden resolverse en núcleos independientes (por ejemplo, calcular en paralelo las bandas horizontales del tablero) y luego integrarse en una etapa de combinación.
- **Programación Dinámica:** Problemas secuenciales por etapas donde múltiples trayectorias de decisión convergen al mismo estado acumulado (optimización de rutas, planificación de inventarios, alineamiento de secuencias o programación dinámica por perfil en tableros).

c. Limitaciones encontradas

- **Explosión exponencial sin poda (Fuerza Bruta):** El crecimiento O(9^n) impide aplicar fuerza bruta pura en cualquier tablero real de Sudoku (n ≥ 40), obligando a establecer un límite de seguridad de 100,000 combinaciones en la comparativa.
- **Óptimos locales sin salida (Greedy):** La imposibilidad de deshacer asignaciones hace que Greedy falle cuando todas las celdas vacías tienen 2 o más candidatos y la primera opción lleva a una contradicción posterior (como ocurrió en el caso extremo de 17 pistas).
- **Costo de permutaciones por fila en tableros muy vacíos (Divide y Vencerás y Programación Dinámica):** Ambos métodos dependen de generar las permutaciones válidas de cada fila (completar_fila). Cuando el tablero supera las 55 celdas vacías (como el caso extremo con filas completamente vacías donde 9! = 362,880 permutaciones por fila), el costo de precalcular y cruzar filas crece rápidamente, razón por la cual estos dos métodos se acotan a tableros con n ≤ 55 celdas vacías.